

Relatório Técnico: Prototipagem do Sistema de Investimentos VcRiquinho

Hálan William da Costa Sousa

Resumo

Este documento apresenta a especificação técnica e o relatório de design para o protótipo do sistema de gestão e alocação de investimentos da startup "VcRiquinho". O projeto foi desenvolvido utilizando os princípios de Programação Orientada a Objetos (POO) em Java, integrando uma arquitetura separada em *model*, *view* e *controller*, em camadas com persistência em banco de dados MySQL via *Stored Procedures*, cobrindo o gerenciamento de clientes (Pessoa Física e Jurídica), contas bancárias com regras de rendimento específicas, e produtos financeiros de renda fixa e variável.

1 Introdução e Contextualização

A startup de investimentos *VcRiquinho* iniciou suas operações com o modelo de negócio voltado à gestão e alocação estratégica de recursos de clientes conforme perfis específicos de investimento. O lucro da empresa obtém-se através de taxas de serviço aplicadas sobre os rendimentos obtidos ao longo de períodos arbitrários. Para validar essa proposta, desenvolveu-se um protótipo integrado de linha de comando conectado a um banco de dados relacional, abrangendo o gerenciamento completo (CRUD) de clientes, contas e produtos financeiros, além da simulação de rendimentos e taxas de serviço. Além disso, como requisito, implementou-se um CRUD para gerenciamento dos produtos de investimento.

2 Decisões de Design (Arquitetura Orientada a Objetos e Banco de Dados)

Para assegurar a manutenibilidade, a robustez e a aderência aos requisitos do projeto, adotou-se uma arquitetura estruturada em pacotes (`model`, `view`, `controller` e `dao`).

- **Hierarquia de Clientes e Contas:** Utilização de classes abstratas e herança para separar `IndividualClient` (PF) e `CorporateClient` (PJ) sob a super-classe `Client`, bem como diferentes modalidades de contas (`CheckingAccount`, `CdiAccount` e `AutoInvestmentAccount`) estendendo `BaseAccount` que implementa a interface `Account`.

- **Polimorfismo para Rendimentos e Taxas:** Aplicação de métodos polimórficos de cálculo de rendimento (`calculateYield`) e taxas de serviço (`calculateServiceFee`) com base na tipagem dinâmica das contas e dos produtos financeiros associados.
- **Persistência com Stored Procedures:** O acesso aos dados isola as regras de negócio do banco de dados relacional `vcRiquinhoDB` por meio de procedimentos armazenados (*Stored Procedures*), garantindo transacionabilidade segura e integridade referencial com restrições `ON DELETE CASCADE`, que garante que os relacionamentos que surgem para adequar o modelo orientado à objetos ao relacional sejam apagados em caso do apagamento do registro no banco.

3 Descrição do Funcionamento do Sistema

O fluxo operacional do sistema contempla as seguintes etapas principais:

1. **Autenticação e Cadastro:** O operador realiza o cadastro de clientes (PF ou PJ) via `ClientController` e procedimento armazenado `CADASTRO_CLIENTE`, ou efetua o *login* populando dinamicamente o objeto `Client` e suas respectivas contas associadas.
2. **Gestão de Contas e Produtos:** O sistema permite a criação de novas contas vinculadas ao cliente e o gerenciamento de produtos de investimento de Renda Fixa (com período de carência) e Renda Variável.
3. **Simulação de Rendimentos:** Através de parâmetros de tempo (30, 60, 90 ou 120 dias), o sistema calcula rendimentos e aplica as taxas específicas (ex.: 0,07% para CDI; 0,1% para PF e 0,15% para PJ em contas de auto-investimento), desconsiderando automaticamente produtos de renda fixa que se encontrem dentro do período de carência.

4 Exemplo de Código do Protótipo

Abaixo encontra-se parte da implementação do controlador `ClientController`, responsável pelo gerenciamento e simulação das regras de negócio das contas:

```

1  public void calcularRendimento(Client cli, String id, int dias)
2  {
3      for (BaseAccount acc : cli.getAccounts()) {
4          if (acc.getId().equalsIgnoreCase(id)) {
5              if (acc instanceof CheckingAccount) {
6                  System.out.println("Rendimento: " +
7                      ((CheckingAccount) acc).calculateYield(dias));
8              } else if (acc instanceof CdiAccount) {
9                  System.out.println("Rendimento: " +
10                     ((CdiAccount) acc).calculateYield(dias));
11             } else if (acc instanceof AutoInvestmentAccount) {
12                 System.out.println("Rendimento: " +
13                     ((AutoInvestmentAccount) acc).calculateYield(dias));
14             }
15         }
16     }
17     return;

```

```

15     }
16 }
17 System.out.println("Conta nao encontrada");
18 }
19

```

Listing 1: Trecho do ClientController.java

5 Modelagem do Banco de Dados e Diagramas

O sistema é sustentado por um esquema relacional normalizado no MySQL (*VcRiquinhoDB*), compreendendo tabelas estruturadas para clientes, especializações PF/PJ, contas bancárias, contas auxiliares (CDI e Auto-Investimento) e produtos financeiros com suas respectivas restrições de renda fixa e variável. Para compreender melhor a modelagem da parte orientada à objetos desse sistema foi desenvolvido um diagrama de classes UML.

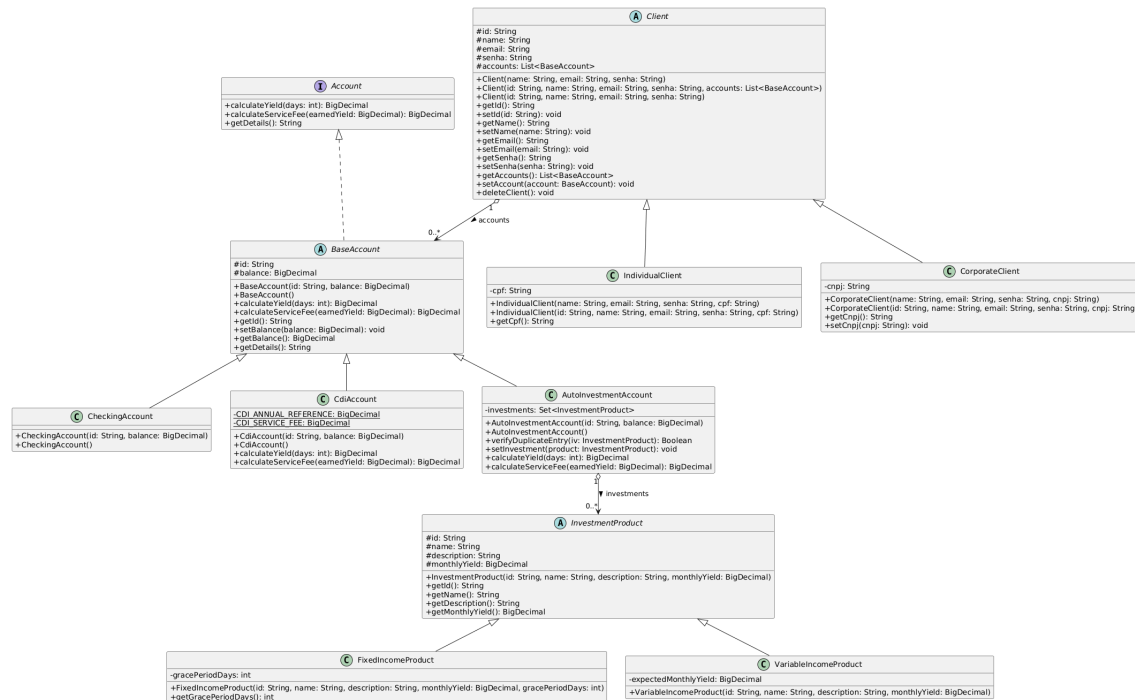


Figura 1: Diagrama UML

6 Considerações Finais

O protótipo desenvolvido cumpre integralmente os requisitos funcionais e técnicos estipulados para a startup *VcRiquinho*. A sinergia entre os conceitos avançados de Orientação a Objetos em Java e a persistência orientada a procedimentos armazenados em banco de dados assegura uma base sólida, extensível e segura para futuras expansões comerciais e operacionais da plataforma.